Lecture Notes on Deep Neural Networks

Jawwad Ahmed Shamsi
FAST–NUCES, Pakistan

April 7, 2026

1 KL Divergence (Intuition and Terms)

In Variational Autoencoders (VAEs), **KL Divergence** measures how different the learned latent distribution is from a standard normal distribution.

$$D_{KL}(q_\phi(z|x) \,||\, p(z)) = -\frac{1}{2}\sum_{j=1}^{k}\left(1 + \log(\sigma_j^2) - \mu_j^2 - \sigma_j^2\right)$$

The goal is to make the learned distribution $q_\phi(z|x)$ as close as possible to:

$$p(z) = \mathcal{N}(0, 1)$$

1.1 Explanation of Terms

- $q_\phi(z|x)$: The distribution learned by the encoder. Instead of producing a single value, the encoder outputs a mean (μ) and variance (σ^2).
- $p(z)$: The target distribution, which is a standard normal distribution with mean 0 and variance 1.
- $\sum_{j=1}^{k}$: KL divergence is computed for each dimension of the latent vector and then summed.
- μ_j: Mean of the learned distribution in dimension j. The term $-\mu_j^2$ penalizes the model if the mean moves away from 0.
- σ_j^2: Variance of the learned distribution. The term $-\sigma_j^2$ penalizes large variance.
- $\log(\sigma_j^2)$: This term prevents the variance from collapsing to very small values and stabilizes learning.
- Constant (1): A balancing term from the Gaussian formulation.
- $-\frac{1}{2}$: A scaling factor that ensures the correct magnitude of KL divergence.

1.2 Intuition

KL divergence acts as a **soft constraint** on the latent space. It encourages:

- $\mu \to 0$
- $\sigma^2 \to 1$

This ensures that the latent space becomes smooth, continuous, and well-structured.

1.3 Simple Example

Good case:

$$\mu = 0,\ \sigma = 1 \Rightarrow KL \approx 0$$

Bad case:

$$\mu = 3,\ \sigma = 0.1 \Rightarrow KL \text{ is large}$$

Key Idea: KL divergence acts as a regularizer that keeps the latent space smooth and Gaussian.

2 Reparameterization Trick

In a Variational Autoencoder, the encoder outputs a distribution:

$$q_\phi(z|x) = \mathcal{N}(\mu, \sigma^2)$$

From this distribution, we need to sample a latent vector z.

2.1 The Problem

Sampling is a random operation and is **not differentiable**. This prevents backpropagation through the network.

2.2 The Solution: Reparameterization Trick

We rewrite the sampling process as:

$$z = \mu + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, 1)$$

2.3 Explanation of Terms

- μ: Mean predicted by the encoder
- σ: Standard deviation predicted by the encoder
- ϵ: Random noise sampled from a standard normal distribution
- z: Latent vector passed to the decoder

2.4 Why This Works

Instead of sampling directly from a changing distribution, we:

- Sample from a fixed distribution $\mathcal{N}(0, 1)$
- Shift and scale it using μ and σ

This moves the randomness into ϵ, while keeping μ and σ differentiable. As a result, gradients can flow through the network.

2.5 Step-by-Step Process

1. Input x is passed to the encoder
2. Encoder outputs μ and σ
3. Sample $\epsilon \sim \mathcal{N}(0, 1)$
4. Compute:
$$z = \mu + \sigma \cdot \epsilon$$
5. Pass z to the decoder

2.6 Intuition

Instead of sampling from a changing distribution, we sample from a fixed distribution and adjust it using learned parameters. This makes the model trainable and stable.

2.7 Simple Example

$$\mu = 2, \quad \sigma = 3, \quad \epsilon = 0.5$$

$$z = 2 + 3 \times 0.5 = 3.5$$

Key Idea: Reparameterization allows gradient flow through stochastic sampling by expressing it as a deterministic function of μ, σ, and noise.